Budapesti Műszaki SZC Bláthy Ottó Titusz Informatikai Technikum

VIZSGAREMEK

Készült: 2026.04.20.

Budapesti Műszaki SZC Bláthy Ottó Titusz Informatikai Technikum

VIZSGAREMEK

Stead-E

Szokás követő életmód alkalmazás

Konzulens:
Dr. Szabó Richárd Péter
Gavallér György

Készítette: Tavas Tamara
Kaba Nóra Rebeka
Dudás Balázs

Hallgatói nyilatkozat

Alulírottak, ezúton kijelentjük, hogy a vizsgaremek saját, önálló munkánk, és korábban még sehol nem került publikálásra.

| --- | --- | --- |
| Tavas Tamara | Kaba Nóra Rebeka | Dudás Balázs |

Konzultációs lap

Vizsgázók neve: Tavas Tamara, Kaba Nóra Rebeka, Dudás Balázs
Vizsgaremek címe: Stead-E
Program nyújtotta szolgáltatások:

- Platformfüggetlen
- Regisztráció kezelése
- Szokás követés
- Célok kítűzése, kezelése
- Teljesítmény nyomon követése
- Statisztikák
- Eredmények vizuális követése
- Felhasználói beállítások
- Admin felület
- Ütemezett szokások

A vizsgaremek beadható:
Budapest, 2026. .....
.....
Konzulens

A vizsgaremeket átvettem:
Budapest, 2026. .....
.....
A szakképzést folytatóintézmény felelőse

Tartalom

I. rész: Felhasználói Dokumentáció 5
1. A projektről 5
1.1 Mi a STEAD-E? 5
1.2 Kik készítették? 5
1.3 A motiváció mögötte 6
2. Rendszerkövetelmények 6
2.1 Webböngészős verzió 6
2.2 Android alkalmazás 6
3. Első lépések 7
3.1 Regisztráció 7
3.3 Profil beállítása 8
..... 8
4. Az alkalmazás funkciói 9
4.1 Főoldal (Dashboard) 9
4.2 Szokások kezelése 10
4.2.1 Szokás hozzáadása 10
4.2.2 Szokás szerkesztése 10
..... 11
4.2.3 Szokás törlése 11
4.2.4 Szokás teljesítése 11
4.3 Célok kezelése 12
4.3.1 Cél létrehozása 12
4.3.2 Cél szerkesztése és törlése 12
4.3.3 Haladás követése 13
4.4 Statisztikák 14
4.5 Eredmények (Achievementek) 15
4.6 Profiloldal 16
5. Admin funkciók 17
5.1 Felhasználók kezelése 17
5.2 Jogosultságok kezelése 17
..... 18
II. rész: Fejlesztői Dokumentáció 18
1. Technikai áttekintés 18

1.1 A két fejlesztési ág 18
1.2 Architektúra 18
2. Laravel webalkalmazás 19
2.1 Technológiai stack 19
2.2 Fejlesztői környezet beállítása 19
2.4 Adatbázis 21
2.5 Backend: Controllerek és logika 24
2.6 API endpointok 25
2.7 Webes útvonalak 26
2.8 Hitelesítés és jogosultságkezelés 27
2.9 Frontend: Blade, JS, CSS 27
2.10 Tesztelés 29
2.10.1 Backend tesztek (`Laravel / PHPUnit`) 29
2.10.2 Android tesztek (Kotlin / JUnit) 31
2.10.3 Postman API tesztek 32
3. Android alkalmazás 34
3.1 Technológiai stack 34
3.2 Fejlesztői környezet beállítása 34
3.3 API kommunikáció 34
3.4 Hitelesítés kezelése 35
4. Közreműködők 35

Stead-E

I. rész: Felhasználói Dokumentáció

---

1. A projektről

1.1 Mi a STEAD-E?

A STEAD-E egy egészséges életmódot támogató, szokáskövető alkalmazás, amely elérhető böngészőből és Android okostelefonról egyaránt. A névben a "Steady" szó rejtőzik (állhatatosság, kitartás), de egyszerre utal a Wall-E filmbéli robotra is ez az összefüggés nem véletlen. Az alkalmazás nem a teljesítményversenyre, hanem a következetességre és a tartós, egészséges szokások kialakítására helyezi a hangsúlyt.

Az ötlet lényege egyszerű: a mai digitális világ tele van olyan technológiákkal, amelyek mozgásszegény életmódra csábítanak, és az alkalmazás pont ezt akarja ellensúlyozni nem tiltással, hanem folyamatos, barátságos ösztönzéssel.

1.2 Kik készítették?

A projektet egy háromfős csapat fejlesztette szakmai záróvizsga keretében:

Név	Szerep	Felelősségi területek
Tavas Tamara	Frontend Designer	UI/UX tervezés, dokumentáció
Dudás Balázs	Backend fejlesztő	Laravel API, adatbázis, webes frontend
Kaba Nóra Rebeka	Mobilfejlesztő	Android (Kotlin) alkalmazás, API integráció

1.3 A motiváció mögötte

Az alkalmazás mögött az a felismerés áll, hogy az emberek nagy részének nem az hiányzik, hogy *tudja*, mit kellene csinálnia (mozogni, eleget aludni, vizet inni), hanem az, hogy rendszeresen meg is tegye. A STEAD-E ebben nyújt segítséget: láthatóvá teszi az előrehaladást, jutalmazza a következetességet, és sorozatokban (streak) méri az aktivitást így a felhasználó napról napra motivált marad.

---

2. Rendszerkövetelmények

2.1 Webböngészős verzió

A webes verzió bármilyen modern böngészőben működik, külön telepítés nélkül. Javasolt böngészők:

- Google Chrome (97-es verzió vagy újabb)
- Mozilla Firefox (95-ös verzió vagy újabb)
- Microsoft Edge (97-es verzió vagy újabb)
- Safari (15-ös verzió vagy újabb)

Internet kapcsolat szükséges. Az alkalmazás reszponzív, tehát mobilböngészőből is kényelmesen használható, bár a dedikált Android alkalmazás jobb élményt nyújt mobil eszközökön.

2.2 Android alkalmazás

- Android 8.0 (Oreo) vagy újabb operációs rendszer
- Legalább 50 MB szabad tárhely a telepítéshez
- Internetkapcsolat szükséges az adatok szinkronizálásához
- Az alkalmazás APK formátumban terjeszthető, nem szükséges Play Store hozzáférés

---

3. Első lépések

3.1 Regisztráció

Az alkalmazás használatához regisztráció szükséges. A regisztrációhoz az alábbi adatokat kell megadni:

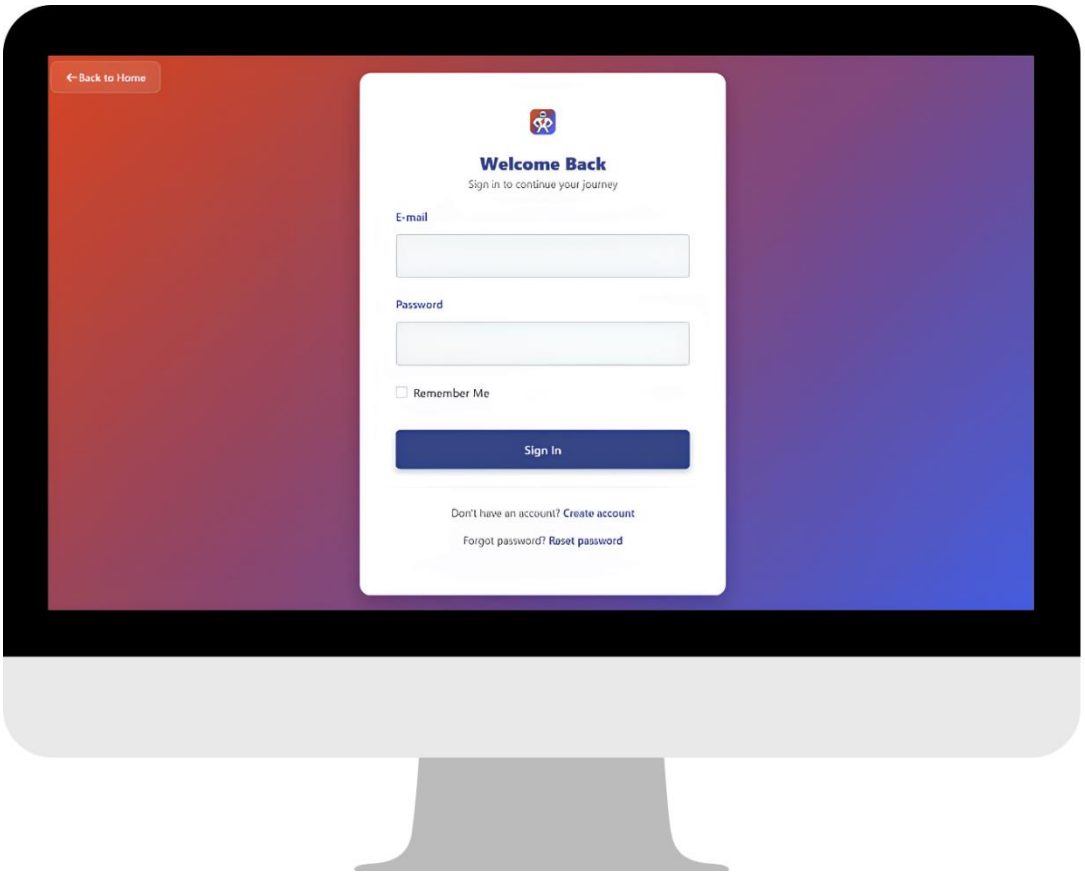
1. **Teljes név**: A megjelenítéshez használt név
2. **Felhasználónév**: Egyedi azonosító, amellyel bejelentkezés is lehetséges
3. **E-mail cím**: Érvényes e-mail cím, amely szintén egyedi kell legyen
4. **Jelszó**: Minimum 8 karakter hosszúságú jelszó, amelyet megerősítéssel kell megadni

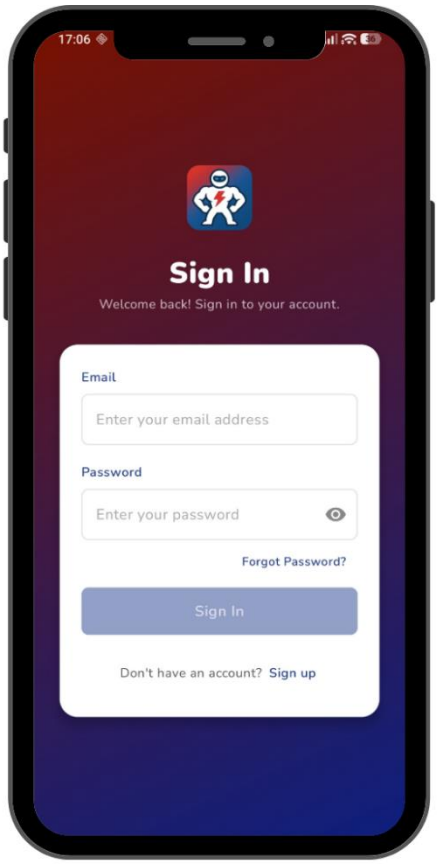
A regisztrációs űrlap kitöltése után a rendszer automatikusan bejelentkezteti a felhasználót, és átirányítja a főoldalra (dashboardra). Az első belépéskor az alkalmazás rögtön felkínálja a profil kiegészítését is.

Megjegyzés: A jelszó titkosítva tárolódik, azt senki a fejlesztők sem láthatják.

3.2 Bejelentkezés

A bejelentkezési felületen az e-mail cím vagy a felhasználónév megadásával, valamint a jelszóval lehet belépni. Hibás adatok esetén az alkalmazás hibaüzenetet jelenít meg. Sikeres bejelentkezés után a felhasználó a főoldalra kerül.



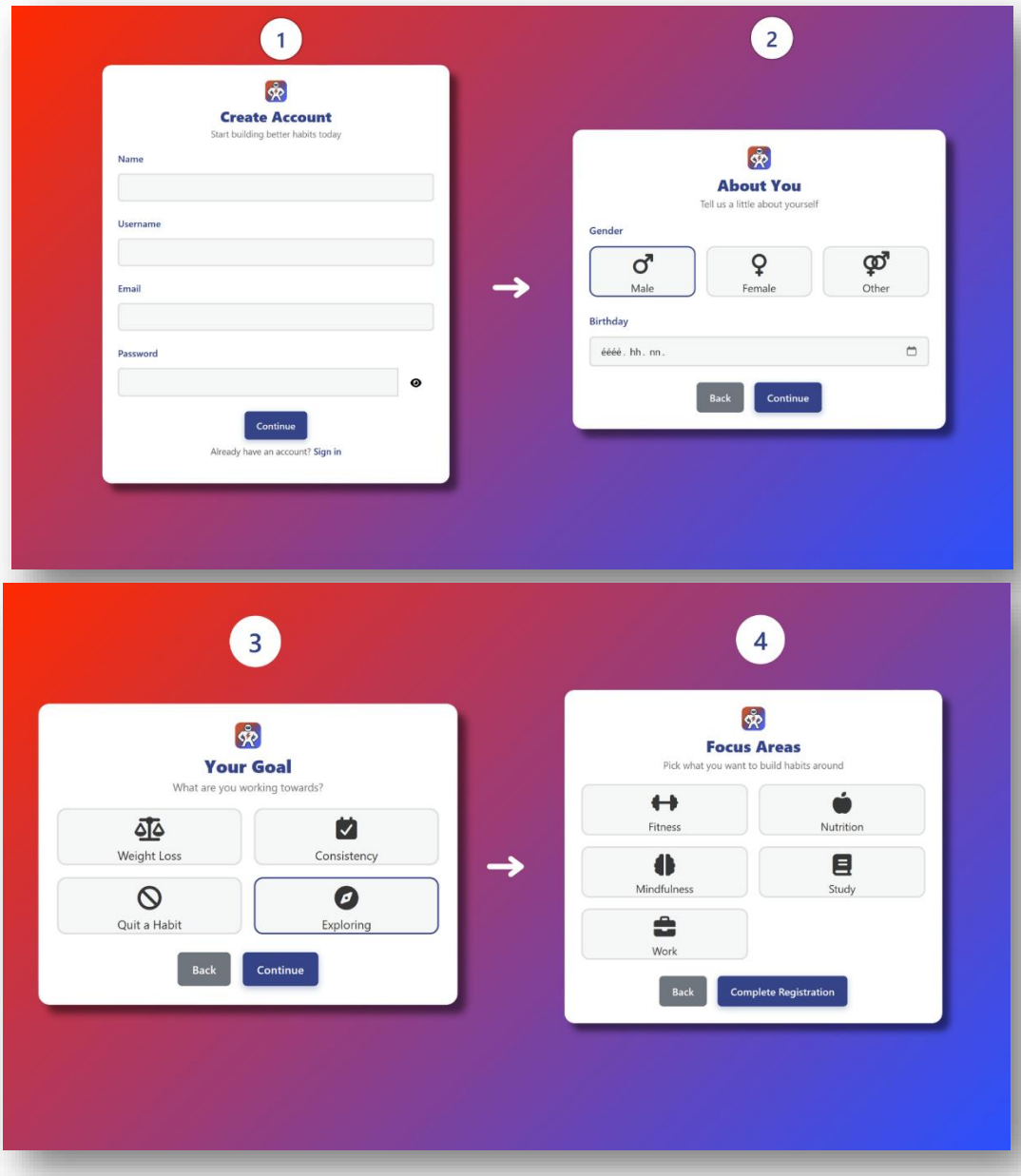


3.3 Profil beállítása

A regisztráció elvégzése után érdemes kiegészíteni a profilt, mert néhány statisztika és ajánlás pontosabb lesz, ha az alkalmazás ismeri az alapadatokat. A profiloldalon az alábbiak állíthatók be:

- **Nem** (biológiai nem, opcionális)
- **Születési dátum** (az életkor kiszámításához)
- **Preferált szokáskategóriák**: amelyek iránt leginkább érdeklődik a felhasználó (pl. sport, táplálkozás, mentális egészség)
- **Profilkép**: feltölthető saját fotó

Ezek az adatok bármikor módosíthatók, és a törlésükre is van lehetőség.

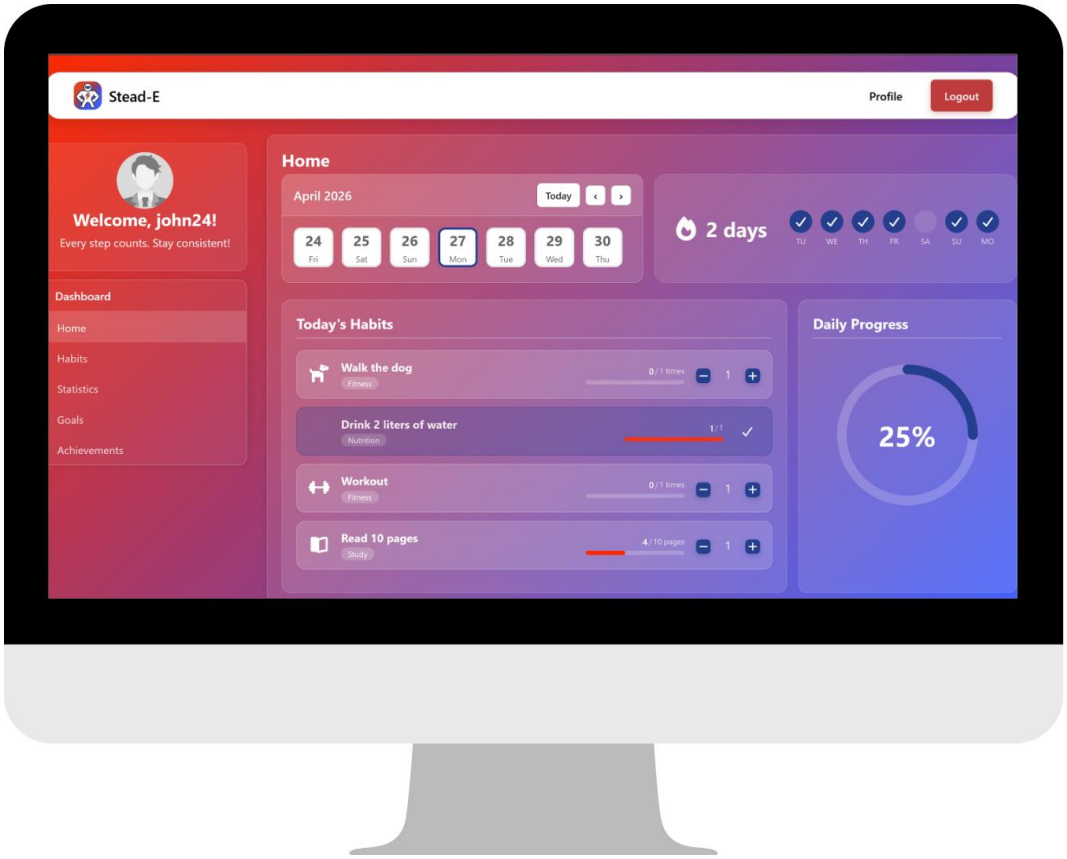


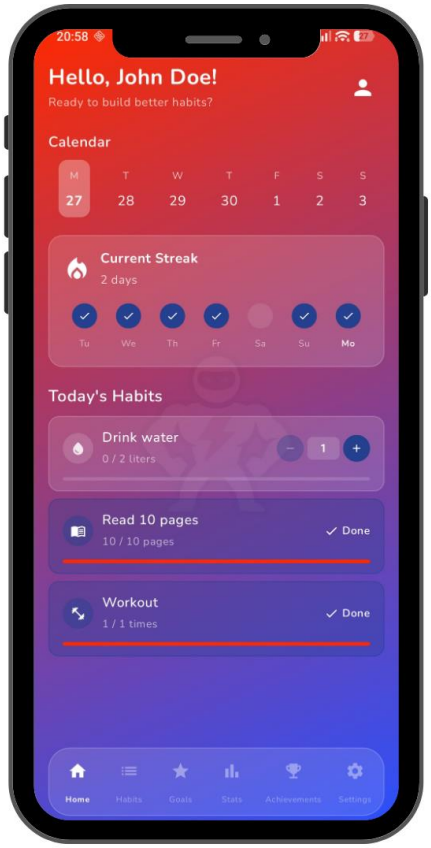
4. Az alkalmazás funkciói

4.1 Főoldal (Dashboard)

A főoldal az alkalmazás szíve. Ide kerül a felhasználó minden bejelentkezés után, és innen tekinthető meg egy pillantással a napi előrehaladás.
A főoldalon a következők láthatók:

- **Mai szokások listája**: Az aznapi elvégzendő szokások, mellettük a teljesítés gombbal. Szokásonként látható, hogy hány egységet kell teljesíteni és hányat sikerült eddig.
- **Jelenlegi sorozat (streak)**: Hány egymást követő napon teljesítette a felhasználó legalább az egyik szokását. Ez az egyik legfontosabb motiváló jelzőszám.
- **Heti vizualizáció**: Az elmúlt 7 nap teljesítési előzménye, vizuálisan megjelenítve, hogy mely napokon volt aktív a felhasználó.
- **Dátum szerinti böngészés**: A főoldalon lehetőség van korábbi napok megtekintésére is, így visszamenőlegesen is ellenőrizhető az előrehaladás.





4.2 Szokások kezelése

A szokások képezik az alkalmazás alapját. Minden olyan ismétlődő tevékenység szokásként rögzíthető, amelyet a felhasználó rendszeresen el akar végezni.

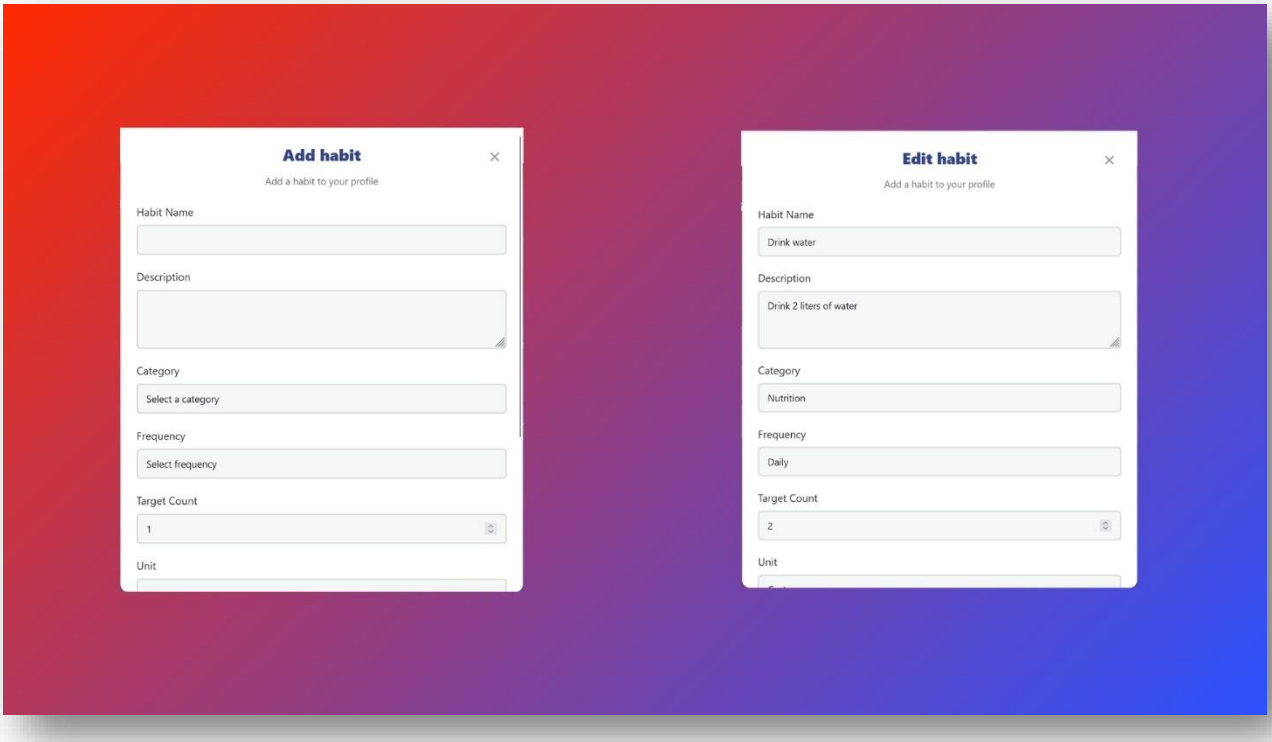
4.2.1 Szokás hozzáadása

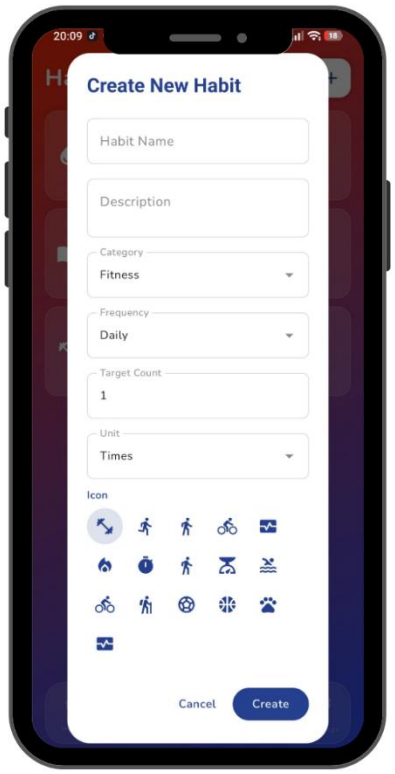
Új szokás létrehozásakor az alábbi adatokat kell megadni:

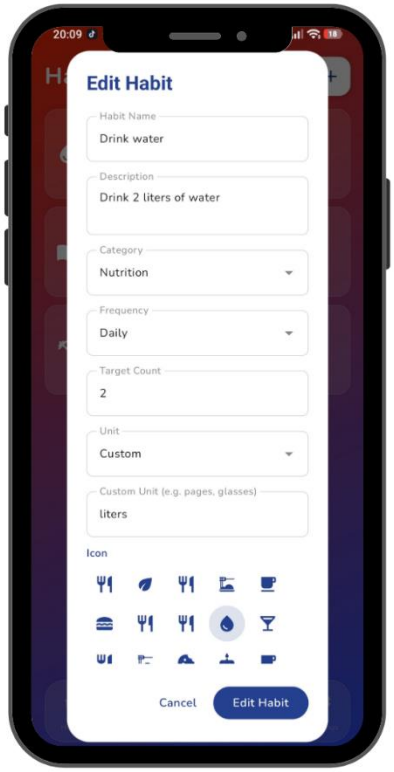
- **Megnevezés**: A szokás neve (pl. "Reggeli futás", "2 liter víz ivása")
- **Leírás**: Opcionális részletezés
- **Kategória**: A szokás besorolása (pl. sport, táplálkozás, mentális egészség, alvás)
- **Ikon**: Vizuális megjelenítés az ikonkönyvtárból
- **Ismétlési gyakoriság**: Megadható, hogy a szokást naponta, hetente, havonta vagy egyedi napokon kell elvégezni
- **Ütemezett napok**: Heti vagy havi szokásoknál megadható, hogy a hét/hónap melyik napjaira szóljon
- **Célmennyiség és mértékegység**: Ha a szokás mérhető (pl. 8 pohár víz, 30 perc séta), megadható a célszám és a mértékegysége
- **Kapcsolódó cél**: A szokás opcionálisan hozzárendelhető egy korábban létrehozott célhoz

4.2.2 Szokás szerkesztése

Bármely korábban létrehozott szokás szerkeszthető. A módosítás azonnal érvénybe lép. Ha a szokás gyakorisága megváltozik, az alkalmazás automatikusan kezeli az ütemezett napok frissítését is.







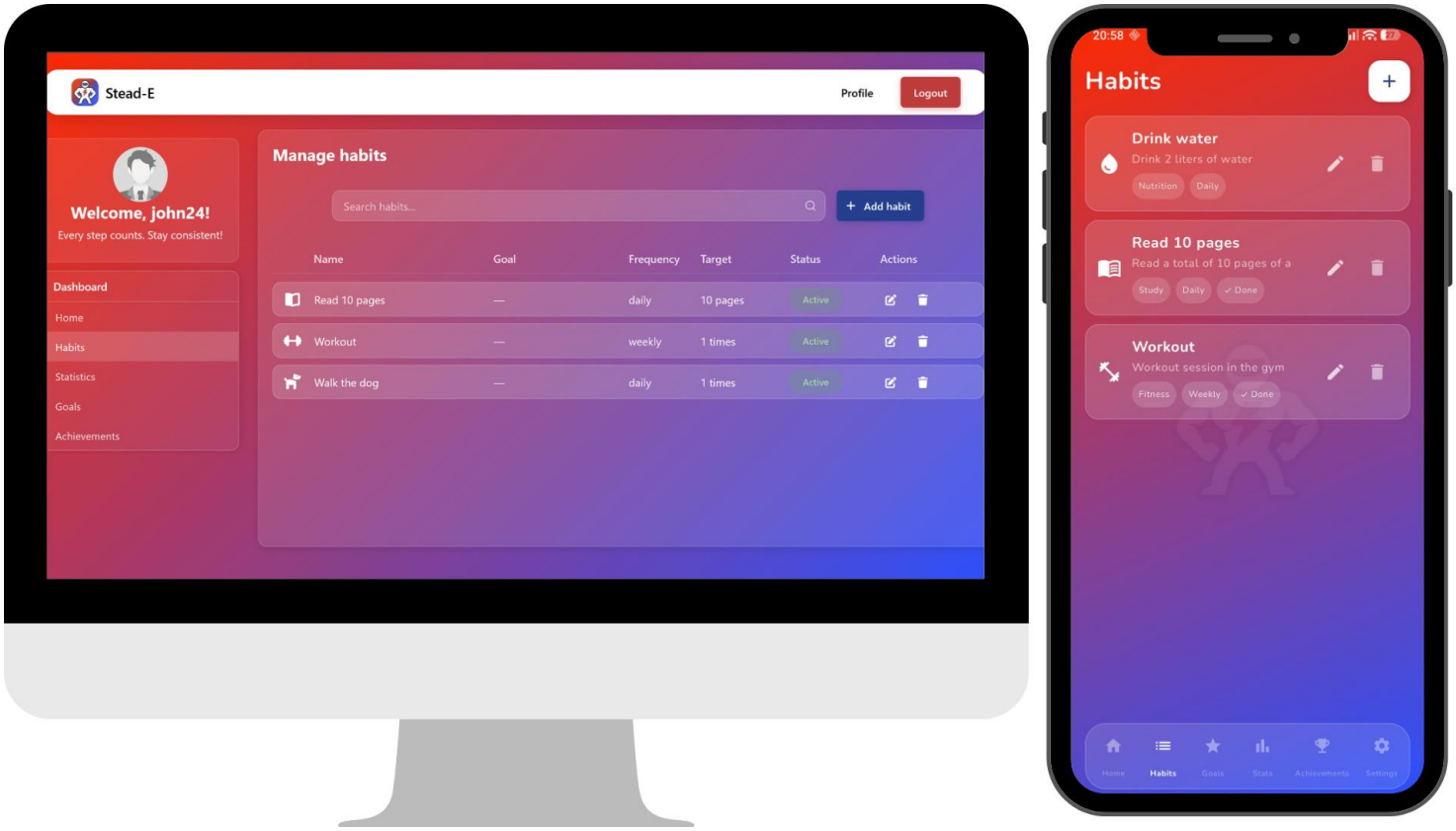
4.2.3 Szokás törlése

Törlés esetén a szokás és a hozzá tartozó előzmények is eltűnnek a listából. A szokások "puha törlés" (soft delete) mechanizmussal kerülnek el. Ez azt jelenti, hogy az adatok az adatbázisban technikailag megmaradnak, de a felhasználó számára nem láthatók.

4.2.4 Szokás teljesítése

A főoldalon az egyes szokások mellett "Teljesítve" gomb jelenik meg. Egy szokás teljesítésekor:

- A rendszer rögzíti a teljesítés időpontját
- Ha a szokásnak van darabszámos célmennyisége, többször is rá lehet nyomni (pl. 8 pohár vízből minden pohár külön teljesítés)
- Opcionálisan megadható: hangulat (mood), megjegyzés a teljesítéshez
- A teljesítés visszavonható ("Visszavonás" gomb), amely az utolsó rögzített teljesítési egységet törli
- Ha a szokás egy célhoz van rendelve, a teljesítés automatikusan növeli a cél előrehaladási értékét
- Az achievementek (eredmények) automatikusan frissülnek minden teljesítésnél



4.3 Célok kezelése

A célok hosszabb távú törekvések, amelyekhez szokások rendelhetők. Egy cél lehet például "Félmaraton futása", amihez a "Napi futás" szokás rendelhető hozzá.

4.3.1 Cél létrehozása

Új cél létrehozásakor szükséges megadni:

- **Cím**: A cél neve
- **Leírás**: Részletezés (opcionális)
- **Kategória**: Besorolás
- **Ikon**: Vizuális ikon
- **Mértékegység**: Amiben a haladás mérhető (pl. km, nap, alkalom)
- **Célérték**: A befejezéshez szükséges teljes mennyiség
- **Határidő**: Meddig szeretné elérni a felhasználó a célt

4.3.2 Cél szerkesztése és törlése

A célok bármikor szerkeszthetők és törölhetők. A törlés a hozzárendelt szokásokat nem érinti, azok megmaradnak, a kapcsolat azonban megszűnik.

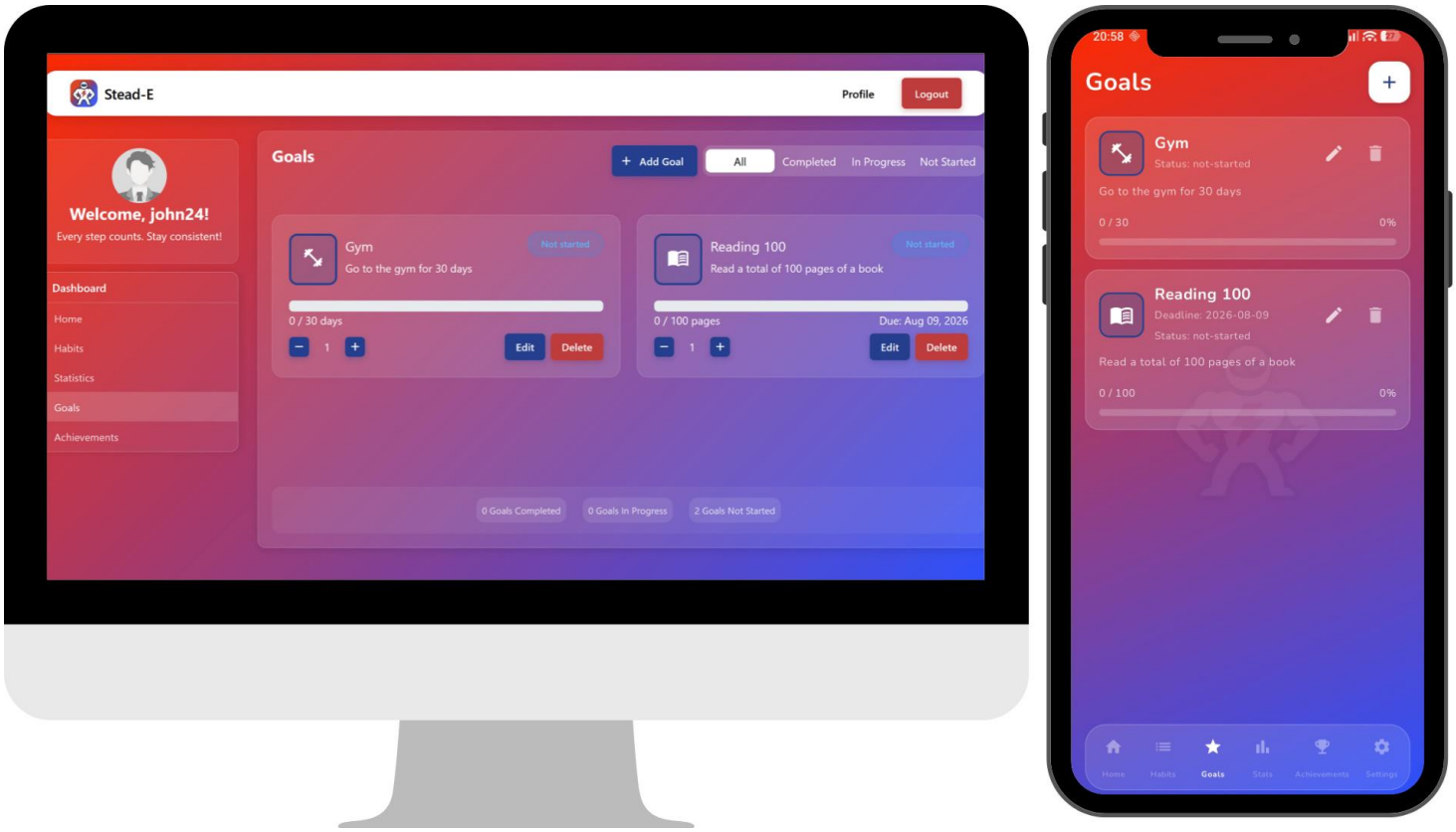
4.3.3 Haladás követése

A célok oldalán minden célnál látható:

- Az aktuális előrehaladás a célértékhez viszonyítva (pl. 14/42 km)
- A cél státusza: Nem elkezdett / Folyamatban / Befejezve
- A határidőig hátralévő idő
- A cél előrehaladása százalékosan is megjelenik

A haladás kétféleképpen frissülhet:

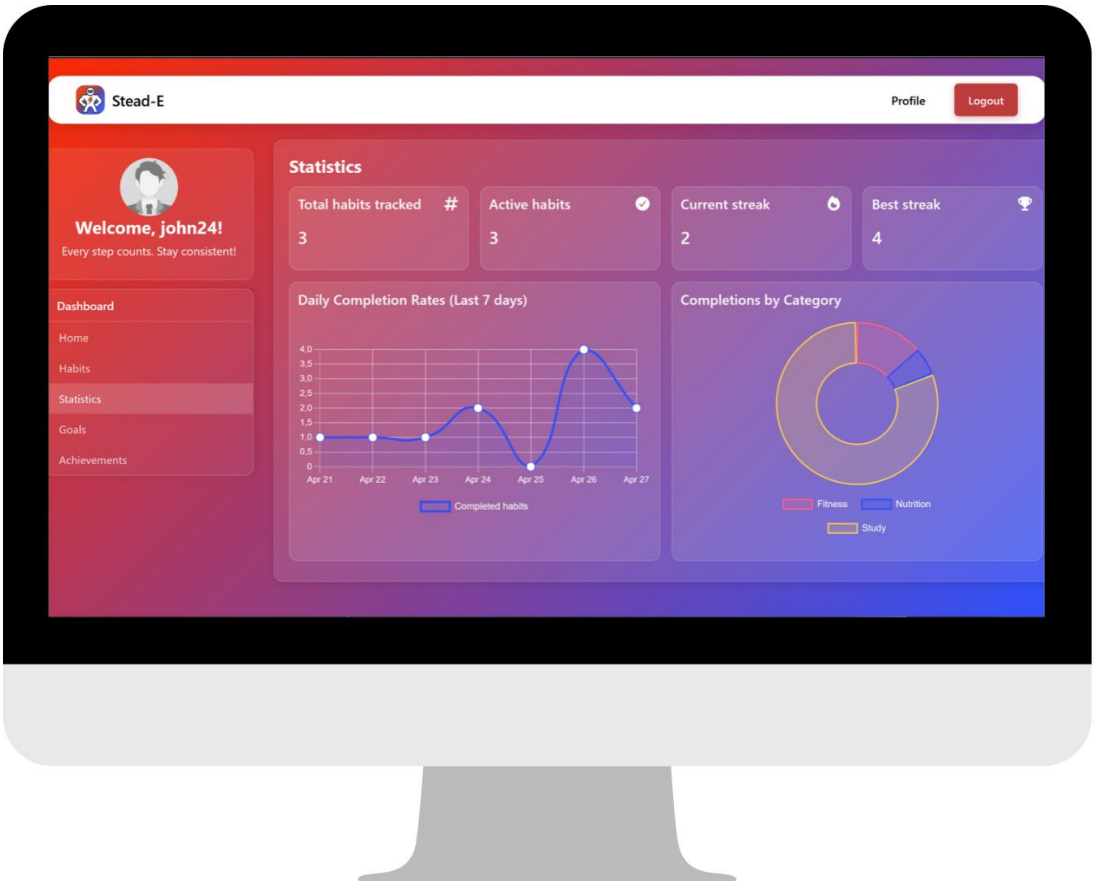
1. Automatikusan, ha az adott célhoz rendelt szokást teljesíti a felhasználó
2. Manuálisan, a célhoz rögzített közvetlen haladásrögzítéssel

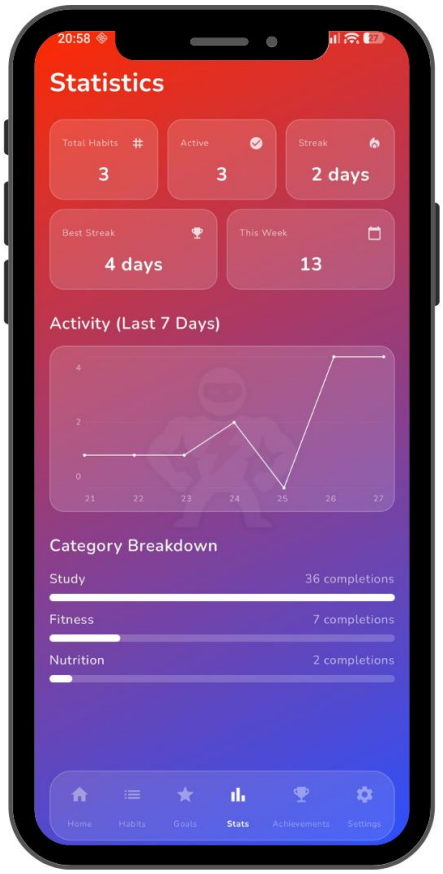


4.4 Statisztikák

A statisztikák oldalon az összesített adatok böngészhetők, ami segít megérteni a saját szokásmintákat és fejlődést.
Az oldalon megjelenő adatok:

- **Jelenlegi sorozat**: Hány nap óta teljesít minden napra legalább egy szokást
- **Leghosszabb sorozat**: Az eddigi leghosszabb megszakítás nélküli aktív időszak
- **Napi teljesítések száma**: Összesítve, mennyi szokást teljesített eddig összesen
- **Kategóriák szerinti bontás**: Melyik kategóriában mennyit teljesített arányosan (pl. sport: 40%, táplálkozás: 35%)
- **Heti összesítő**: Az elmúlt hetek aktív napjainak alakulása



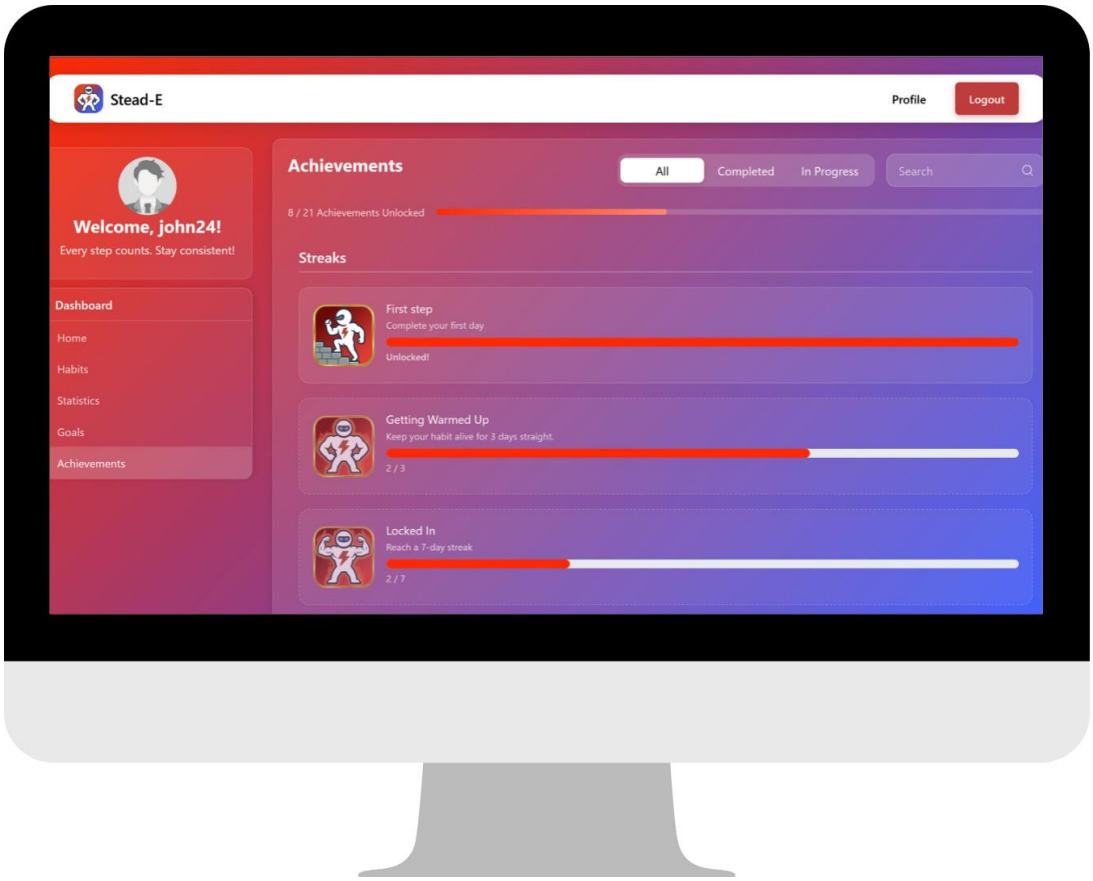


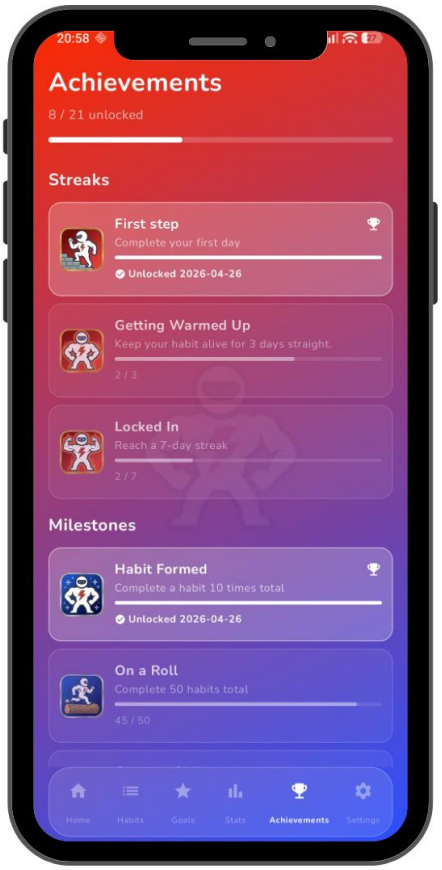
4.5 Eredmények (Achievementek)

Az achievementek (eredmények, kitüntetések) egy gamification elemet hoznak az alkalmazásba amivel elismerik és megjutalmazzák a folyamatos szokás követést. Ezek automatikusan nyílnak fel, amikor a felhasználó elér egy-egy mérföldkövet.
Az achievementek három típusba sorolhatók:

1. **Sorozat achievementek**: A megszakítás nélküli napok számához kötöttek (pl. 7 napos sorozat, 30 napos sorozat, 100 napos sorozat)
2. **Mérföldkő achievementek**: Összesített teljesítési számokhoz kötöttek (pl. 10., 50., 100. teljesítés)
3. **Kategória achievementek**: Adott kategóriában (pl. sport, alvás) elért összes teljesítéshez kötöttek

Az achievementek oldalán minden achievement látható, a már feloldottak és a még el nem értek egyaránt. A zárolt achievementeknél megjelenik az is, hogy mennyire sikerült eddig közelíteni a feloldáshoz (százalékos haladás).





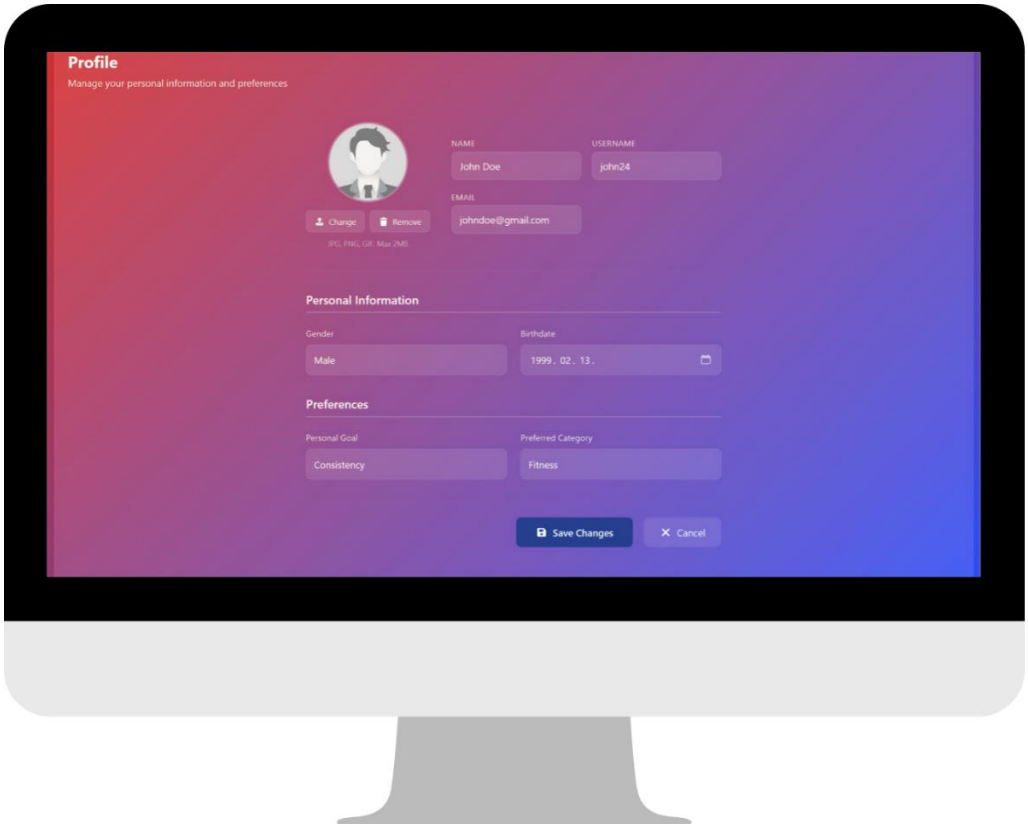
4.6 Profiloldal

A profiloldalon a felhasználói adatok szerkeszthetők, és itt kezelhető a profilkép is.

Módosítható adatok:

- Név és felhasználónév
- E-mail cím
- Biometrikus adatok (nem, születési dátum, magasság, testsúly)
- Alvási idők (lefekvés, ébredés)
- Preferált szokáskategóriák
- Profilkép (feltöltés és törlés)

A jelszó módosítása szintén elvégezhető a profiloldalon viszont a régi jelszó megadása szükséges hozzá.



5. Admin funkciók

Az alkalmazásban létezik egy adminisztrátori szerepkör. Az adminok hozzáférnek egy külön admin panelhez, amely a normál felhasználók számára nem látható.

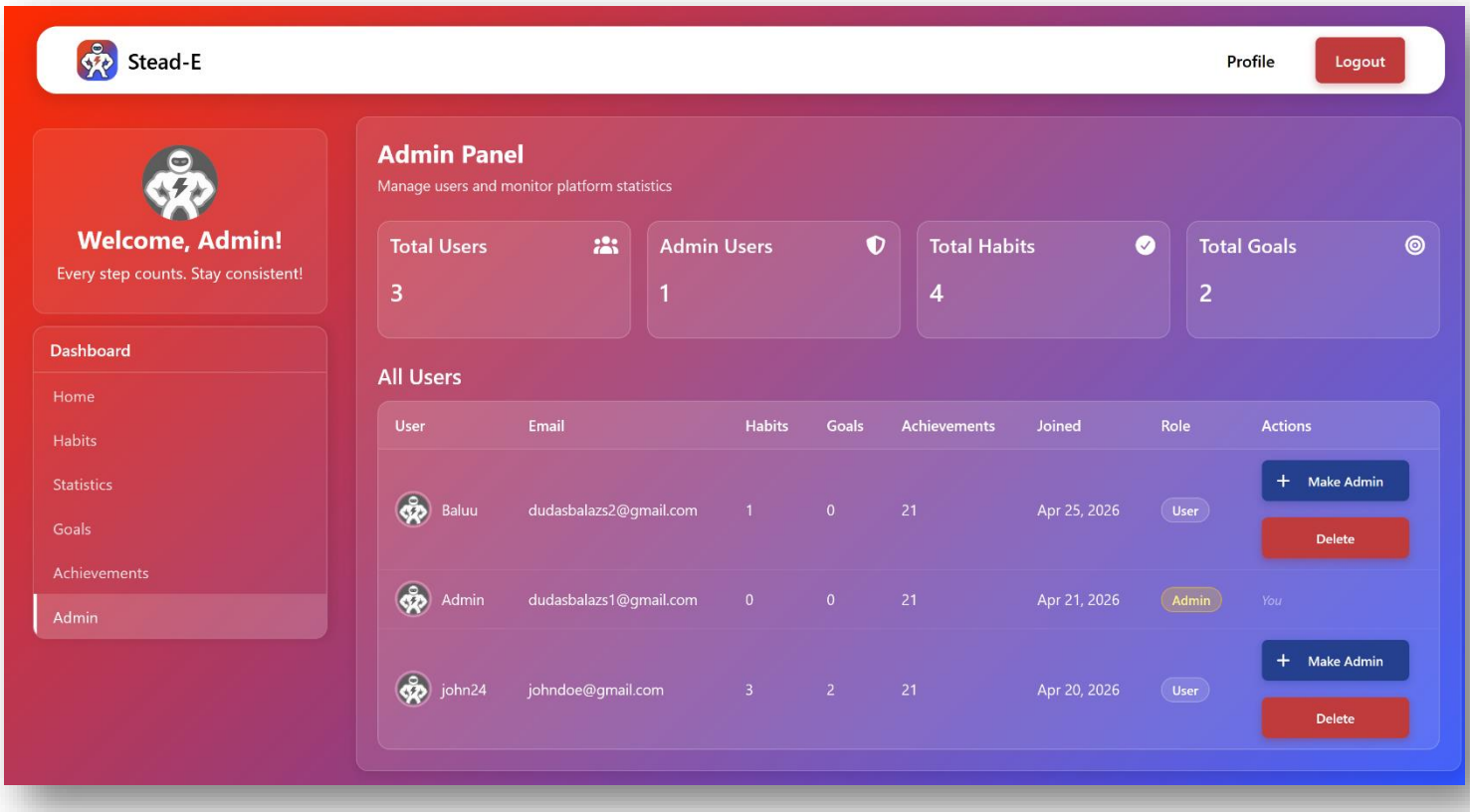
5.1 Felhasználók kezelése

Az admin panel főbb funkciói:

- **Felhasználók listája**: Az összes regisztrált felhasználó neve, e-mail címe, szokásainak és céljainak száma
- **Felhasználó törlése**: Egy felhasználó és az összes hozzá tartozó adata törölhető (visszafordíthatatlan művelet, az admin saját magát nem törölheti)
- **Statisztikák**: Az admin látja a rendszer egészének összesített adatait: összes felhasználó, szokás, cél és adminok száma

5.2 Jogosultságok kezelése

Az adminok más felhasználókat adminná emelhetnek ("promote"), illetve az admin jogosultságot vissza is vonhatják ("demote"). Az admin saját magát nem demotálhatja, ez a funkció le van tiltva, hogy ne fordulhasson elő véletlenül, hogy admin nélkül marad a rendszer.



Stead-E

II. rész: Fejlesztői Dokumentáció

---

1. Technikai áttekintés

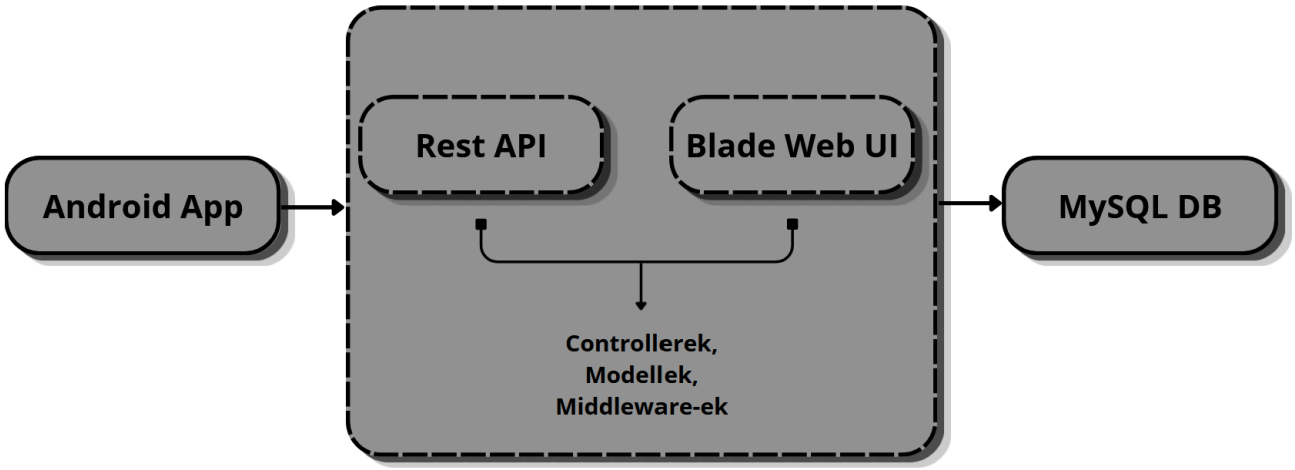
1.1 A két fejlesztési ág

A STEAD-E projekt fejlesztői oldalról két önálló alkalmazásból áll, amelyek egymástól független kódbázison futnak, de egyazon REST API-n keresztül kommunikálnak.

Az első a **Laravel webalkalmazás**, amely kettős szerepet tölt be: egyrészt ez maga a backend szerver, amely az összes üzleti logikát, adatbázis-műveleteket és az API-t tartalmazza; másrészt itt él a böngészős webes felület is, amelyet a Laravel Blade sablon motora rendereค. Ez a kódbázis egyetlen repóban van, és a két réteget (API + web) ugyanazok a controllerek szolgálják ki, content negotiation segítségével.

A második a **natív Android alkalmazás**, amelyet Kotlin nyelven fejlesztett a csapat. Ez egy teljesen önálló projekt, saját Android Studio-s struktúrával, a backendhez való kapcsolódás kizárólag az API-n keresztül valósul meg.

1.2 Architektúra



A controllerek az `$request->expectsJson()` és az is `('api/*')` ellenőrzés alapján döntik el, hogy JSON választ adjanak-e vissza (mobil kliensnek) vagy Blade nézetet rendereljenek (böngészőnek). Így ugyanaz a logika mindkét klienst kiszolgálja, kódduplikáció nélkül.

2. Laravel webalkalmazás

2.1 Technológiai stack

Komponens	Technológia	Verzió
Nyelv	PHP	8.2+
Framework	Laravel	12
Adatbázis	MySQL	8.0
Hitelesítés (API)	Laravel Sanctum	4.0
Hitelesítés (web)	Laravel UI / Session	-
Template motor	Laravel Blade	-
CSS framework	Bootstrap	5.2
Build eszköz	Vite	7.0
Diagram/chart	Chart.js	4.5
Ikonok	Font Awesome	-
PHP csomagkezelő	Composer	-
JS csomagkezelő	npm	-
Tesztelés	PHPUnit	11

2.2 Fejlesztői környezet beállítása

Előkövetelmények:

- PHP 8.2 vagy újabb
- Composer
- Node.js 18+ és npm
- Git

Telepítés lépései:

1. *Klónozás*

   `git clone https://github.com/baluu25/project-stead-e.git`

   `cd project-stead-e`

2. *PHP függőségek telepítése*

   `composer install`

   JavaScript függőségek telepítése

   `npm install`

3. *Adatbázis migrációk és seederek futtatása*

   `php artisan migrate --seed`

4. *Storage link létrehozása (profilképekhez)*

   `php artisan storage:link`

5. *Frontend build (fejlesztői módban, két terminálban párhuzamosan)*

   `npm run dev`

   `php artisan serve`

   Az alkalmazás a `http://localhost:8000` címen érhető el.

2.3 Könyvtárstruktúra

```
project-stead-e/
├── app/
│   ├── Http/
│   │   ├── Controllers/         ← Request handlerek
│   │   │   ├── AuthController.php
│   │   │   ├── HabitController.php
│   │   │   ├── HabitCompletionController.php
│   │   │   ├── GoalController.php
│   │   │   ├── HomeController.php
│   │   │   ├── StatisticsController.php
│   │   │   ├── UserController.php
│   │   │   ├── AchievementController.php
│   │   │   ├── AdminController.php
│   │   │   └── Auth/            ← Laravel UI auth controllerek
│   │   └── Middleware/
│   │       └── AdminMiddleware.php
│   ├── Models/
│   │   ├── User.php
│   │   ├── Habit.php
│   │   ├── HabitCompletion.php
│   │   ├── Goal.php
│   │   └── Achievement.php
│   ├── Policies/
│   └── Providers/
│       └── AppServiceProvider.php
│
├── database/
│   ├── migrations/              ← Táblák séma definíciói
│   ├── factories/               ← Tesztadatok gyártói
│   └── seeders/                 ← AchievementSeeder
│
├── routes/
│   ├── api.php                  ← /api/* végpontok (Sanctum)
│   └── web.php                  ← Webes útvonalak (session auth)
│
├── resources/
│   ├── views/                   ← Blade sablonok
│   └── js/                      ← Frontend JavaScript
│
├── public/                      ← Webszerver gyökér
├── storage/                     ← Feltöltött fájlok, logok
├── tests/
│   ├── Unit/
│   └── Feature/
├── docs/                        ← Projektdokumentáció, képek
├── composer.json
├── package.json
└── vite.config.js
```

2.4 Adatbázis

Főbb táblák:

Users

- id, name, username (unique), email (unique), password (bcrypt)
- gender, birthdate, weight, height, profile_picture
- sleep_time, wake_time, user_goal
- preferred_categories (JSON)
- is_admin (boolean, default: false)
- timestamps

Habits

- id
- user_id → FK: users.id (cascade delete)
- goal_id → FK: goals.id (nullable, set null on delete)
- name, description, category, icon
- frequency → enum: daily | weekly | monthly | custom
- scheduled_days →JSON (nullable, heti/havi szokásoknál kitöltött)
- target_count, unit
- is_active → boolean
- deleted_at → (soft delete)
- timestamps

Habit_completions

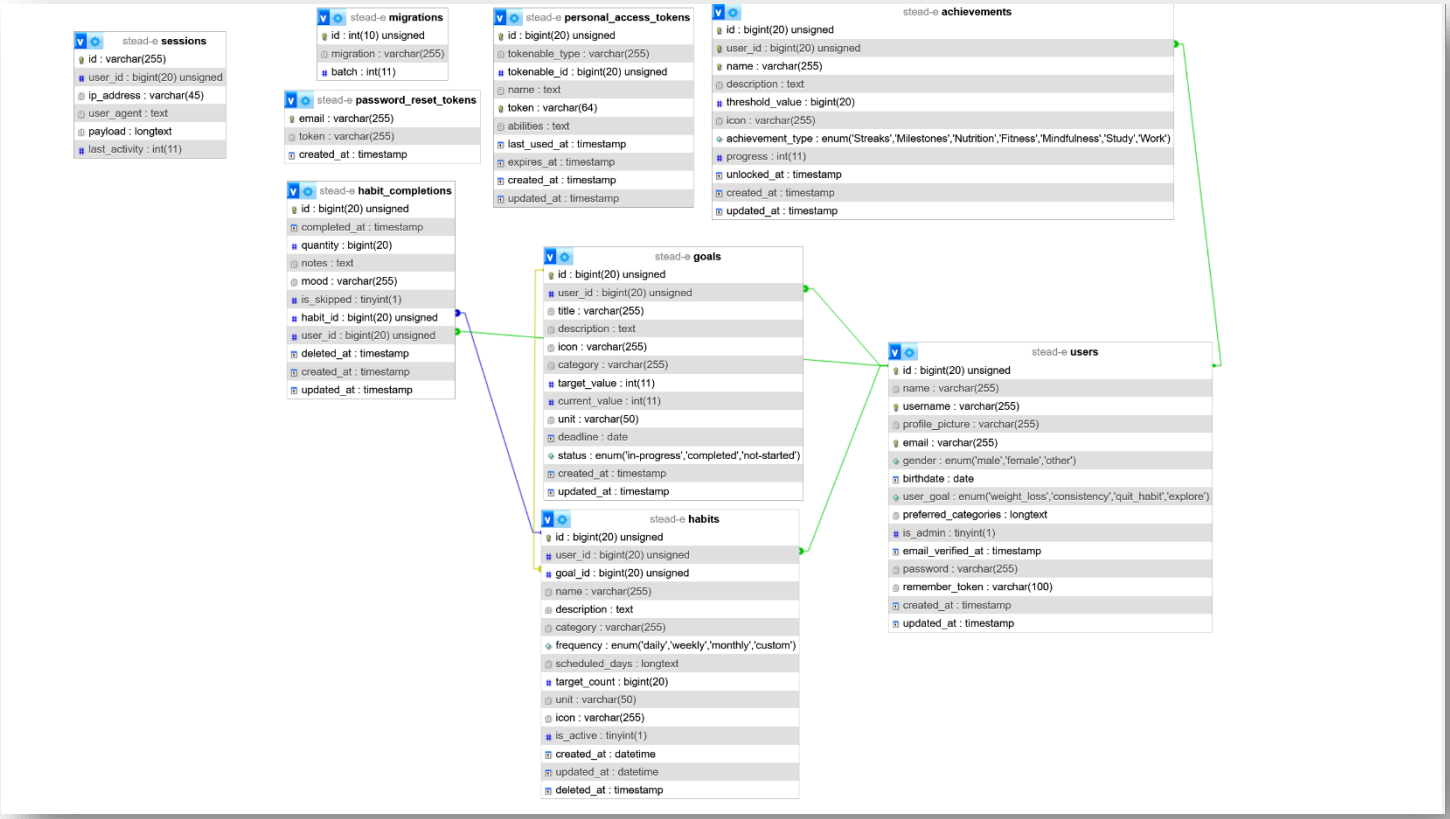
- id
- habit_id → FK: habits.id (cascade delete)
- user_id → FK: users.id (cascade delete)
- completed_at → timestamp
- quantity → integer (alapértelmezett: 1)
- notes, mood
- is_skipped → boolean (kihagyás jelölése)
- deleted_at →(soft delete)
- timestamps

Goals

- id
- user_id → FK: users.id (cascade delete)
- title, description, icon, category, unit
- target_value → decimal
- current_value → decimal (alapértelmezett: 0)
- deadline → date (nullable)
- status → enum: not-started | in-progress | completed
- timestamps

Achievements

- id
- user_id → FK: users.id (cascade delete)
- name, description, icon
- achievement_type varchar (Streaks / Milestones / kategórianév)
- threshold_value integer (mekkora értéknél nyílik fel)
- progress → integer
- unlocked_at → timestamp (nullable. Null, ha még nem nyílt fel)
- timestamps



Adatbázis kapcsolatok

Modellek és kapcsolatok

Modell	Főbb kapcsolatok	Egyéb
User	hasMany: Habit, HabitCompletion, Goal, Achievement	HasApiTokens (Sanctum)
Habit	belongsTo: User, Goal, hasMany: HabitCompletion	SoftDeletes, scheduled_days JSON cast
HabitCompletion	belongsTo: Habit, User	SoftDeletes
Goal	belongsTo: User, hasMany: Habit	progressPercentage() = számított érték
Achievement	belongsTo: User	syncForUser() statikus metódus, progressPercentage()

Az `Achievement::syncForUser(User $user)` metódus minden szokásteljesítésnél meghívódik, majd ellenőrzi az összes achievement feltételét, és feloldja azokat, amelyek elérik a küszöbértéket.

Migrációk futtatása

- Táblák létrehozása és seederek futtatása: `php artisan migrate --seed`
- Fejlesztés alatt, ha mindent elölről kell kezdeni: `php artisan migrate:fresh -seed`

2.5 Backend: Controllerek és logika

AuthController

Kezeli a regisztrációt, bejelentkezést és kijelentkezést. Regisztrációkor validált adatokkal hozza létre a usert, majd meghívja az AchievementSeeder-t, hogy az achievement rekordok rögtön bekerüljenek. Bejelentkezéskor Sanctum tokent generál, amit a válaszban visszaad a kliensnek. Web session-alapú flow-hoz a Laravel UI által generált Auth\LoginController stb. felelnek.

HabitController

A szokások listájánál (apiIndex / index) az aktív szokások mellé csatolja az adott napi teljesítési adatokat is. Szokás létrehozásánál és módosításánál a frequency értékétől függően kezeli a scheduled_days JSON mezőt, a napi szokásnál ez null marad, heti/havi szokásnál a megadott napok kerülnek bele.

HabitCompletionController

Teljesítés rögzítésekor a következő lépések zajlanak le:

1. Ellenőrzi, hogy a habit valóban a bejelentkezett userhez tartozik
2. Létrehozza a HabitCompletion rekordot (quantity, notes, mood mezőkkel)
3. Ha a habithez cél van rendelve, növeli a Goal.current_value értékét a teljesített mennyiséggel
4. Frissíti a cél státuszát (in-progress / completed)
5. Meghívja az Achievement::syncForUser()-t

Visszavonáskor az amount paraméter alapján törli az utolsó N soft-delete-elhető rekordot, és visszacsökkenti a goal haladását.

GoalController

CRUD műveletek a célokhoz. A progress action közvetlen haladásrögzítést biztosít (nem szokáson keresztül). A cél státuszát automatikusan frissíti: ha current_value >= target_value, completed-re állítja; ha current_value > 0, in-progress-re.

HomeController

A dashboard adatait állítja össze. A ?date=YYYY-MM-DD paraméter alapján szűri a szokásokat: napi szokásoknál minden nap megjelenik, heti szokásoknál a scheduled_days tömbben lévő hétköznapokon, havi szokásoknál a megadott napokon. Kiszámolja az aktuális streak-et és összeállítja a 7 napos vizualizációs adathalmazt.

StatisticsController

Az összes teljesítési dátumot sorrendbe rendezi, majd megszámlálja az egymást követő napokat. Ebből kerül ki az aktuális és a valaha volt leghosszabb sorozat. Kategóriánkénti bontást is számol.

UserController

Profil lekérés és módosítás. Képfeltöltésnél a fájlt a storage/app/public/profile_pictures mappába menti, frissíti az adatbázist, és törli a régi képet, ha volt.

AchievementController

Listázza az achievementeket, és minden lekérés előtt meghívja a syncForUser()-t, hogy naprakész adatokat adjon vissza. A webIndex metódus Blade nézetet renderel, az index metódus JSON-t ad vissza.

AdminController

Adminisztrátori műveletek. Beépített önvédelmi logika van benne: az admin sem saját magát nem törölheti, sem saját admin jogát nem vonhatja vissza. Ezek az esetek explicit hibaüzenettel visszautasítódnak.

2.6 API endpointok

Minden API útvonal a /api prefix alatt van. A védett végpontokhoz Authorization: Bearer {token} fejléc szükséges.

Nyilvános (autentikáció nélkül):

Metódus	Útvonal	Leírás
POST	/api/register	Regisztráció
POST	/api/login	Bejelentkezés, tokent ad vissza

Védett (`auth:sanctum`):

Metódus	Útvonal	Leírás
POST	/api/logout	Kijelentkezés (token törlése)
GET	/api/user	Bejelentkezett user adatai
GET	/api/habits	Szokáslista (napi teljesítésekkel)
POST	/api/habits	Új szokás
PUT	/api/habits/{id}	Szokás módosítása
DELETE	/api/habits/{id}	Szokás törlése
POST	/api/habit-completions	Teljesítés rögzítése
DELETE	/api/habit-completions/{habitId}/today/last	Utolsó teljesítés visszavonása

GET	/api/goals	Célok listája
POST	/api/goals	Új cél
PUT	/api/goals/{goal}	Cél módosítása
DELETE	/api/goals/{goal}	Cél törlése
POST	/api/goals/{goal}/progress	Közvetlen haladásrögzítés
GET	/api/home	Dashboard adatok (?date=YYYY-MM-DD)
GET	/api/statistics	Statisztikák
GET	/api/achievements	Achievementek
GET	/api/profile	Profil részletek
PUT	/api/profile	Profil frissítése
POST	/api/profile/picture	Profilkép feltöltése

Admin (`auth:sanctum` + `admin middleware`):

Metódus	Útvonal	Leírás
GET	/api/admin/users	Felhasználók listája
DELETE	/api/admin/users/{user}	Felhasználó törlése
PATCH	/api/admin/users/{user}/promote	Admin jogosultság adása
PATCH	/api/admin/users/{user}/demote	Admin jogosultság elvétele

2.7 Webes útvonalak

A böngészős felület session-alapú auth-ot használ (auth middleware). Az útvonalak web.php-ban vannak definiálva.

Metódus	Útvonal	Leírás
GET /	welcome.blade.php	Kezdőlap
GET /index	index.blade.php	Landing page
GET /contact	contact.blade.php	Kapcsolat
GET /home	HomeController@index	Dashboard
GET /habits	HabitController@index	Szokások

POST /habit-completions	HabitCompletionController@store	Teljesítés (web form)
GET /statistics	StatisticsController@index	Statisztikák
GET /goals	GoalController@index	Célok
GET /achievements	AchievementController@webIndex	Achievementek
GET /profile	GET /profile	Profil
PUT /profile	UserController@update	Profil mentése
GET /admin	AdminController@index	Admin panel

A bejelentkezési és regisztrációs útvonalakat az `Auth::routes()` hívás generálja automatikusan (Laravel UI).

2.8 Hitelesítés és jogosultságkezelés

API: Laravel Sanctum (token-alapú)

Bejelentkezéskor a backend egy plainTextToken-t generál, amelyet a mobilalkalmazás eltárol és minden kérésnél elküld Authorization: Bearer {token} fejlécben. Kijelentkezéskor a token törlődik az adatbázisból.

Web: Session alapú

A böngészős forgalomhoz a Laravel beépített session-alapú hitelesítése szolgál, amelyet a Laravel UI csomag által generált controllerek kezelnek. A csrf-token automatikusan csatolódik minden form kéréshez.

Admin middleware (AdminMiddleware)

Az admin aliassal regisztrált middleware ellenőrzi a users.is_admin mezőt. Ha az értéke false, a kérés 403 Forbidden válasszal elutasítódik, illetve JSON-ban API kérések esetén, redirect-tel webes kérések esetén.

Erőforrás-szintű jogosultság-ellenőrzés minden controllerben egyedileg van megírva: az aktuálisan bejelentkezett user ID-ját összehasonlítja az erőforrás user_id mezőjével, és eltérés esetén 403-at ad vissza.

2.9 Frontend: Blade, JS, CSS

Blade sablonok (resources/views/):

Két fő layout létezik:

- layouts/app.blade.php: publikus oldalakhoz (landing, login, register)
- layouts/dashboard_app.blade.php: bejelentkezett felhasználók oldalaihoz, navigációs sávval

Az oldalak ezeket terjesztik ki a @extends és @section direktívákkal.

JavaScript fájlok (resources/js/):

Fájl	Feladat
app.js	Fő belépési pont
bootstrap.js	Axios alapbeállítás, CSRF token csatolása
api.js	API hívások absztrakciós rétege
home.js	Dashboard interakciók (teljesítés gomb, dátumváltás)
habits.js	Szokáslista, CRUD modálok
goals.js	Célok CRUD, haladásrögzítés
statistics.js	Chart.js alapú statisztikai vizualizáció
achievements.js	Achievement kártyák megjelenítése
profile.js	Profil szerkesztés, képfeltöltés előnézet
login.js	Bejelentkezési form kezelés
register.js	Regisztrációs form kezelés

CSS és stílusok:

Bootstrap 5.2 adja az alap reszponzív layout rendszert és az UI komponenseket (kártyák, modálok, gombok). A Tailwind CSS 4.0 utility osztályok finomhangolásra, egyedi elrendezésekre és szín-overridekre szolgálnak. Sass (.scss) fájlok is helyet kaptak a custom stílusoknak.

A build folyamat:

- npm run dev
- npm run build

2.10 Tesztelés

2.10.1 Backend tesztek (`Laravel / PHPUnit`)

A tesztek a tests/ könyvtárban találhatók, PHPUnit 11 alapon. A tesztelési konfiguráció SQLite in-memory adatbázist használ, így a fejlesztői adatbázist nem érinti.

Unit tesztek (`tests/Unit/`)

HabitModelTest.php

Teszt	Mit ellenőriz
habit_belongs_to_user	A szokás helyesen kapcsolódik a felhasználóhoz (belongsTo reláció)
habit_has_many_completions	A szokáshoz több teljesítés is tartozhat (hasMany reláció)
habit_belongs_to_goal	A szokás opcionálisan célhoz rendelhető (belongsTo reláció)
habit_goal_nullable	A goal reláció null értékű lehet, ha nincs cél rendelve
habit_soft_deletable	Törlés esetén soft delete viselkedés, az adatbázisban marad a rekord
scheduled_days_gives_back_array	A scheduled_days mező JSON cast-ként tömböt ad vissza

GoalModelTest.php

Teszt	Mit ellenőriz
progress_percentage_is_correct	A progress attribútum helyesen számolja a százalékot (pl. 25/100 = 25)
progress_maximum_can_be_hundred	A haladás értéke maximum 100 lehet, felülteljesítésnél sem megy tovább
progress_is_zero_if_target_value_is_zero	Ha a célérték nulla, a haladás is nulla (nullával való osztás elkerülése)
goal_connects_to_one_user	A cél helyesen kapcsolódik a felhasználóhoz (belongsTo reláció)
goal_can_connect_to_multiple_users	Egy célhoz több szokás rendelhető (hasMany reláció)

Feature tesztek (tests/Feature/)

HabitControllerTest.php

Teszt	Mit ellenőriz
apiIndex_only_gives_back_own_habits	A lista lekérés csak a bejelentkezett felhasználó szokásait adja vissza
api_index_marks_todays_completed_habits	A mai napon teljesített szokásoknál completed_today = 1 szerepel
api_index_does_not_mark_old_progress	A tegnapi teljesítés nem jelenik meg mai teljesítésként
store_make_new_habit	Új szokás létrehozása sikeres, 201-es választ ad, adatbázisba kerül
store_with_daily_frequency_scheduled_days_should_be_null	Napi frekvenciánál a scheduled_days mező null marad
store_validation_error_when_missing_name	Hiányzó name mező esetén 422-es validációs hiba
store_validation_error_when_category_invalid	Érvénytelen kategória esetén 422-es validációs hiba
store_goal_id_assignible_to_habit	Szokás létrehozásakor goal_id sikeresen rendelhető hozzá
update_modifies_own_habit	PUT kérés sikeresen módosítja a saját szokást

update_cant_modify_other_users_habits	Más felhasználó szokásának módosítása 404-es hibát ad
update_deletes_scheduled_days_if_daily_frequency	Napi frekvenciára váltáskor a scheduled_days törlődik
destroy_deletes_own_habit	Saját szokás törlése soft delete-tel, 204-es választ ad
destroy_does_not_delete_other_users_habits	Más felhasználó szokásának törlése 404-es hibát ad
guest_does_not_access_api_endpoint	Bejelentkezés nélküli hozzáférés 401-es hibát ad

GoalControllerTest.php

Teszt	Mit ellenőriz
api_index_only_gives_back_own_goals	A lista lekérés csak a bejelentkezett felhasználó céljait adja vissza
store_makes_a_new_goal	Új cél létrehozása sikeres, 201-es választ ad, adatbázisba kerül
update_modifies_own_goal	PUT kérés sikeresen módosítja a saját célt
update_does_not_modify_other_user_goal	Más felhasználó céljának módosítása 403-as hibát ad
progress_increases_current_value	A haladásrögzítés növeli a current_value értékét
progress_gets_completed_status_if_reaches_target	Célérték elérésekor a státusz automatikusan completed lesz
progress_does_not_get_above_target_value	A current_value nem haladhatja meg a target_value-t
progress_does_not_modify_other_user_goal	Más felhasználó céljának haladásrögzítése 403-as hibát ad
destroy_deletes_own_goal	Saját cél törlése sikeres, 204-es választ ad
destroy_does_not_delete_other_user_goal	Más felhasználó céljának törlése 403-as hibát ad

Tesztek futtatása

1. Összes teszt
   - `php artisan test`

2. Csak unit tesztek
   - `php artisan test --testsuite=Unit`

3. Csak feature tesztek
   - `php artisan test --testsuite=Feature`

2.10.2 Android tesztek (Kotlin / JUnit)

Az Android alkalmazás unit tesztjei az app/src/test/java/com/TBN/steade/ könyvtárban találhatók. JUnit 4 alapon futnak, eszközt vagy emulátort nem igényelnek, a JVM-en futtathatók közvetlenül Android Studióból.

Unit tesztek

ApiHabitTest.kt

Teszt	Mit ellenőriz
isActive returns true when isActiveRaw is true	Boolean true értéknél az isActive tulajdonság igaz
isActive returns false when isActiveRaw is zero	Nulla számnál az isActive tulajdonság hamis
isActive returns true when isActiveRaw is string 1	"1" szövegnél az isActive tulajdonság igaz
isCompletedToday returns true when completedToday is greater than zero	isCompletedToday returns true when completedToday is greater than zero
isCompletedToday returns false when completedToday is zero	Ha completedToday = 0, az isCompletedToday hamis

ApiGoalTest.kt

Teszt	Mit ellenőriz
progress returns correct ratio	A progress helyesen számolja az arányt (pl. 5/10 = 0.5)
progress is capped at 1 when current exceeds target	A haladás értéke maximum 1.0 lehet, ha a jelenlegi érték meghaladja a célt
progress returns 0 when targetValue is zero	Ha a célérték nulla, a haladás nulla (nullával való osztás elkerülése)
displayName falls back to name when title is null	Ha a title null, a displayName a name mezőt adja vissza

AuthResponseTest.kt

Teszt	Mit ellenőriz
success returns true when token is present	Ha a token jelen van, a success tulajdonság igaz
success returns false when token is null	Ha a token null, a success tulajdonság hamis
success returns false when token is missing entirely	Ha a token teljesen hiányzik a válaszból, a success hamis

Tesztek futtatása Android Studióban

- Jobb klikk a `com.TBN.steade (test)` mappán → Run 'Tests in com.TBN.steade'

2.10.3 Postman API tesztek

Az API végpontok manuális tesztelése Postman segítségével történt. Az alábbi kollekciók és végpontok kerültek tesztelésre:

Auth kollekció

Metódus	Végpont	Leírás
POST	/api/register	Új felhasználó hozzáadása

POST	/api/login	Meglévő felhasználó beléptetése

Goals kollekció

Metódus	Végpont	Leírás
POST	/api/goals	Új cél létrehozása
PUT	/api/goals/{id}	Meglévő cél módosítása
POST	/api/goals/{id}/progress	Haladás rögzítése egy célhoz
DELETE	/api/goals/{id}	Cél törlése

Habits kollekció

Metódus	Végpont	Leírás
GET	/api/habits	Szokások listájának lekérése
POST	/api/habits	Új szokás létrehozása
PUT	/api/habits/{id}	Meglévő szokás módosítása
DELETE	/api/habits/{id}	Szokás törlése

Statistics kollekció

Metódus	Végpont	Leírás
GET	/api/statistics	Statisztikák lekérése

Profile kollekció

Metódus	Végpont	Leírás
GET	/api/profile	Saját profil adatainak lekérése
PUT	/api/profile	Profil adatainak módosítása

Achievements kollekció

Metódus	Végpont	Leírás
GET	/api/achievements	Eredmények listájának lekérése

3. Android alkalmazás

3.1 Technológiai stack

Az Android alkalmazás a STEAD-E önálló mobil kliense, amelyet Kaba Nóra Rebeka fejlesztett. A kód önálló Android Studio projektként él, a Laravel repótól függetlenül.

Komponens	Technológia
Nyelv	Kotlin
Platform	Android 8.0 (Oreo) és újabb
Hálózati kommunikáció	Retrofit
JSON parse	Gson / Moshi
Hitelesítés	Bearer token (SharedPreferences-ben tárolva)
Kommunikáció a backenddel	REST API / JSON

3.2 Fejlesztői környezet beállítása

Az Android alkalmazás fejlesztéséhez szükséges:

- **Android Studio:** (legújabb stabil verzió ajánlott)
- **Android SDK:** legalább API level 26 (Android 8.0)
- **Kotlin plugin:** (Android Studio-ban alapból elérhető)
- **Emulátor:** vagy fizikai Android eszköz az USB debugging bekapcsolásával

A projektet Android Studio-ban a File → Open menüponttal lehet megnyitni. Az első megnyitáskor a Gradle automatikusan letölti a szükséges függőségeket.

A backend API base URL-jét a hálózati konfiguráció fájljában kell beállítani. Fejlesztéshez a Laravel szerver IP-je kell (pl. http://10.0.2.2:8000/api emulátor esetén, ahol a 10.0.2.2 a fejlesztői gép localhost-ja az Android emulátorból nézve)

3.3 API kommunikáció

Az összes hálózati hívást a Retrofit könyvtár kezeli. Az API felületek Kotlin interface-ként vannak definiálva, a Retrofit implementálja őket. Egy hívás például így néz ki:

```
// Habit lista lekérése

interface HabitApiService {

    @GET("habits")

    suspend fun getHabits(): Response<List<Habit>>
```

```
    @POST("habit-completions")
    suspend fun logCompletion(
        @Body body: CompletionRequest
    ): Response<CompletionResponse>
}
```

A Retrofit példányosításakor az Authorization fejléc egy OkHttp Interceptor segítségével kerül automatikusan minden kéréshez csatolva, így ezt a hívási helyen nem kell manuálisan kezelni.

3.4 Hitelesítés kezelése

Bejelentkezéskor a backend által visszaadott token (pl. "1|abc123...") el kell tárolni a SharedPreferences-ben. Kijelentkezéskor a token törlődik a SharedPreferences-ből, és opcionálisan a backend felé is el kell küldeni a POST /api/logout kérést, hogy a szerver oldalon is érvénytelenítse.

A token érvényességét a backend kezeli és ha lejárt vagy érvénytelen tokent küld a mobilalkalmazás, a szerver 401 Unauthorized választ ad vissza, amire az applikációnak a bejelentkezési képernyőre kell navigálnia.

4. Közreműködők

Tavas Tamara

- Frontend Designer
- UI/UX tervezés, dokumentáció

Dudás Balázs

- Backend fejlesztő
- Laravel API, adatbázis, webes frontend

Kaba Nóra Rebeka

- Mobilfejlesztő
- Android (Kotlin) alkalmazás, API integráció

Fejlesztési eszközök:

- GitHub: verziókezelés
- Trello: feladatkezelés
- Discord: csapatkommunikáció
- Postman: API tesztelés

Stead-E | Szakmai záróvizsga projekt, 2026.